

Solution to Problem 2.1 (page 73)

Understanding the relation between hexadecimal and binary formats will be important once we start looking at machine-level programs. The method for doing these conversions is in the text, but it takes a little practice to become familiar.

A. 0x25B9D2 to binary:

Hexadecimal	2	5	B	9	D	2
Binary	0010	0101	1101	1001	1101	0010

B. Binary 1100 1001 0111 1011 to hexadecimal:

Binary	1100	1001	0111	1011
Hexadecimal	C	9	7	B

C. 0xA8B3D to binary:

Hexadecimal	A	8	B	3	D
Binary	1010	1000	1011	0011	1101

D. Binary 11 0010 0010 1101 1001 0110 to hexadecimal:

Binary	11	0010	0010	1101	1001	0110
Hexadecimal	3	2	2	D	9	6

n	2^n (decimal)	2^n (hexadecimal)
5	32	0x20
23	8,388,608	0x800000
15	32,768	0x8000
13	8,192	0x2000
12	4,096	0x1000
6	64	0x40
8	256	0x100

Decimal	Binary	Hexadecimal
0	0000 0000	0x00
$158 = 16 \cdot 9 + 14$	1001 1110	0x9E
$76 = 16 \cdot 4 + 12$	0100 1100	0x4C
$145 = 16 \cdot 9 + 1$	1001 0001	0x91
$16 \cdot 10 + 14 = 174$	1010 1110	0xAE
$16 \cdot 3 + 12 = 60$	0011 1100	0x3C
$16 \cdot 15 + 1 = 241$	1111 0001	0xF1
$16 \cdot 7 + 5 = 117$	0111 0101	0x75
$16 \cdot 11 + 13 = 189$	1011 1101	0xBD
$16 \cdot 15 + 5 = 245$	1111 0101	0xF5

Solution to Problem 2.4 (page 75)

When you begin debugging machine-level programs, you will find many cases where some simple hexadecimal arithmetic would be useful. You can always convert numbers to decimal, perform the arithmetic, and convert them back, but being able to work directly in hexadecimal is more efficient and informative.

- A. $0x605C + 0x5 = 0x6061$. Adding 5 to hex C gives 1 with a carry of 1.
- B. $0x605C - 0x20 = 0x603C$. Subtracting 2 from 5 in the second digit position requires no borrow from the third. This gives 3.
- C. $0x605C + 32 = 0x607C$. Decimal 32 (2^5) equals hexadecimal 0x20.
- D. $0x60FA - 0x605C = 0x9E$. To subtract hex C (decimal 12) from hex A (decimal 10), we borrow 16 from the second digit, giving hex F (decimal 15). In the second digit, we now subtract 5 from hex E (decimal 14), giving decimal 9.

Solution to Problem 2.5 (page 84)

This problem tests your understanding of the byte representation of data and the two different byte orderings.

- A. Little endian: 78 Big endian: 12
- B. Little endian: 78 56 Big endian: 12 34
- C. Little endian: 78 56 34 Big endian: 12 34 56

Recall that `show_bytes` enumerates a series of bytes starting from the one with lowest address and working toward the one with highest address. On a little-endian machine, it will list the bytes from least significant to most. On a big-endian machine, it will list bytes from the most significant byte to the least.

Solution to Problem 2.6 (page 85)

This problem is another chance to practice hexadecimal to binary conversion. It also gets you thinking about integer and floating-point representations. We will explore these representations in more detail later in this chapter.

- A. Using the notation of the example in the text, we write the two strings as follows:

```
0  0  2  7  C  8  F  8
00000000001001111100100011111000
*****
4  A  1  F  2      3      E      0
01001010000111110010001111100000
```

- B. With the second word shifted two positions to the right relative to the first, we find a sequence with 21 matching bits.
- C. We find all bits of the integer embedded in the floating-point number, except for the most significant bit having value 0. Such is the case for the example in the text as well. In addition, the floating-point number has some nonzero high-order bits that do not match those of the integer.

Solution to Problem 2.7 (page 85)

It prints 6D 6E 6F 70 71 72. Recall also that the library routine `strlen` does not count the terminating null character, and so `show_bytes` printed only through the character 'r'.

Solution to Problem 2.8 (page 87)

This problem is a drill to help you become more familiar with Boolean operations.

Operation	Result
a	[01001110]
b	[11100001]
$\sim a$	[10110001]
$\sim b$	[00011110]
$a \& b$	[01000000]
$a \mid b$	[11101111]
$a \wedge b$	[10101111]

Solution to Problem 2.9 (page 89)

This problem illustrates how Boolean algebra can be used to describe and reason about real-world systems. We can see that this color algebra is identical to the Boolean algebra over bit vectors of length 3.

- A. Colors are complemented by complementing the values of R , G , and B . From this, we can see that white is the complement of black, yellow is the complement of blue, magenta is the complement of green, and cyan is the complement of red.
- B. We perform Boolean operations based on a bit-vector representation of the colors. From this we get the following:

$$\begin{array}{llll} \text{Blue (001)} & | & \text{Green (010)} & = \text{Cyan (011)} \\ \text{Yellow (110)} & \& \text{Cyan (011)} & = \text{Green (010)} \\ \text{Red (100)} & \sim & \text{Magenta (101)} & = \text{Blue (001)} \end{array}$$

Solution to Problem 2.10 (page 90)

This procedure relies on the fact that EXCLUSIVE-OR is commutative and associative, and that $a \wedge a = 0$ for any a .

Step	*x	*y
Initially	a	b
Step 1	a	$a \wedge b$
Step 2	$a \wedge (a \wedge b) = (a \wedge a) \wedge b = b$	$a \wedge b$
Step 3	b	$b \wedge (a \wedge b) = (b \wedge b) \wedge a = a$

See Problem 2.11 for a case where this function will fail.

Solution to Problem 2.11 (page 91)

This problem illustrates a subtle and interesting feature of our inplace swap routine.

- A. Both `first` and `last` have value k , so we are attempting to swap the middle element with itself.
- B. In this case, arguments `x` and `y` to `inplace_swap` both point to the same location. When we compute `*x ^ *y`, we get 0. We then store 0 as the middle element of the array, and the subsequent steps keep setting this element to 0. We can see that our reasoning in Problem 2.10 implicitly assumed that `x` and `y` denote different locations.
- C. Simply replace the test in line 4 of `reverse_array` to be `first < last`, since there is no need to swap the middle element with itself.

Solution to Problem 2.12 (page 91)

Here are the expressions:

A. $x \& 0xFF$

B. $x \wedge \sim 0xFF$

C. $x | 0xFF$

These expressions are typical of the kind commonly found in performing low-level bit operations. The expression $\sim 0xFF$ creates a mask where the 8 least-significant bits equal 0 and the rest equal 1. Observe that such a mask will be generated regardless of the word size. By contrast, the expression $0xFFFFFFFF00$ would only work when data type `int` is 32 bits.

<https://www.imperva.com/learn/data-security/data-masking/#:~:text=Data%20masking%20is%20a%20way,sales%20demos%2C%20or%20software%20testing.>

Solution to Problem 2.13 (page 92)

These problems help you think about the relation between Boolean operations and typical ways that programmers apply masking operations. Here is the code:

```
/* Declarations of functions implementing operations bis and bic */
int bis(int x, int m);
int bic(int x, int m);

/* Compute x|y using only calls to functions bis and bic */
int bool_or(int x, int y) {
    int result = bis(x,y);
    return result;
}

/* Compute x^y using only calls to functions bis and bic */
int bool_xor(int x, int y) {
    int result = bis(bic(x,y), bic(y,x));
    return result;
}
```

The `bis` operation is equivalent to Boolean OR—a bit is set in `z` if either this bit is set in `x` or it is set in `m`. On the other hand, `bic(x, m)` is equivalent to `x & ~m`; we want the result to equal 1 only when the corresponding bit of `x` is 1 and of `m` is 0.

Given that, we can implement `|` with a single call to `bis`. To implement `^`, we take advantage of the property

$$x \wedge y = (x \& \sim y) \mid (\sim x \& y)$$

Solution to Problem 2.14 (page 93)

This problem highlights the relation between bit-level Boolean operations and logical operations in C. A common programming error is to use a bit-level operation when a logical one is intended, or vice versa.

Expression	Value	Expression	Value
<code>a & b</code>	0x44	<code>a && b</code>	0x01
<code>a b</code>	0x57	<code>a b</code>	0x01
<code>~a ~b</code>	0xBB	<code>!a !b</code>	0x00
<code>a & !b</code>	0x00	<code>a && ~b</code>	0x01

Solution to Problem 2.15 (page 93)

The expression is `!(x ^ y)`.

That is, $x \hat{=} y$ will be zero if and only if every bit of x matches the corresponding bit of y . We then exploit the ability of `!` to determine whether a word contains any nonzero bit.

There is no real reason to use this expression rather than simply writing `x == y`, but it demonstrates some of the nuances of bit-level and logical operations.

Solution to Problem 2.16 (page 94)

This problem is a drill to help you understand the different shift operations.

x		a << 2		Logical a >> 3		Arithmetic a >> 3	
Hex	Binary	Binary	Hex	Binary	Hex	Binary	Hex
0xD4	[11010100]	[01010000]	0x50	[00011010]	0x1A	[11111010]	0xFA
0x64	[01100100]	[10010000]	0x90	[00001100]	0x0C	[11101100]	0xEC
0x72	[01110010]	[11001000]	0xC8	[00001110]	0x0E	[00001110]	0x0E
0x44	[01000100]	[00010000]	0x10	[00001000]	0x08	[11101000]	0xE9

- `int64_t` is a Standard C++ type for a signed integer of exactly 64 bits. `int64` is not a standard type. The first C++ standard didn't have fixed-width types.
- Fixed-width integers are integral types with a fixed number of bits. The C++ standard only specifies a minimum byte count for types such as `short`, `int` and `long`. Fixed-width integers guarantee a specific size, but their use can have an impact on portability, since they are not supported by all platforms.
- C99 defined a set of **fixed-width integers** (in the `stdint.h` header) that are guaranteed to be the same size on any architecture.

Solution to Problem 2.17 (page 101)

In general, working through examples for very small word sizes is a very good way to understand computer arithmetic.

The unsigned values correspond to those in Figure 2.2. For the two's-complement values, hex digits 0 through 7 have a most significant bit of 0, yielding nonnegative values, while hex digits 8 through F have a most significant bit of 1, yielding a negative value.

Hexadecimal	Binary		
$\vec{x}$		$B2U_4(\vec{x})$	$B2T_4(\vec{x})$
0xA	[1010]	$2^3 + 2^1 = 10$	$-2^3 + 2^2 = -6$
0x1	[0001]	$2^0 = 1$	$2^0 = 1$
0xB	[1011]	$2^3 + 2^1 + 2^0 = 11$	$-2^3 + 2^1 + 2^0 = -5$
0x2	[0010]	$2^1 = 2$	$2^1 = 2$
0x7	[0111]	$2^2 + 2^1 + 2^0 = 7$	$2^2 + 2^1 + 2^0 = 7$
0xC	[1100]	$2^3 + 2^2 = 12$	$-2^3 + 2^2 = -4$

Solution to Problem 2.18 (page 105)

For a 32-bit word, any value consisting of 8 hexadecimal digits beginning with one of the digits 8 through f represents a negative number. It is quite common to see numbers beginning with a string of f's, since the leading bits of a negative number are all ones. You must look carefully, though. For example, the number 0x8048337 has only 7 digits. Filling this out with a leading zero gives 0x08048337, a positive number.

4004d0:	48 81 ec e0 02 00 00	sub	\$0x2e0,%rsp	A. 736
4004d7:	48 8b 44 24 a8	mov	-0x58(%rsp),%rax	B. -88
4004dc:	48 03 47 28	add	0x28(%rdi),%rax	C. 40
4004e0:	48 89 44 24 d0	mov	%rax,-0x30(%rsp)	D. -48
4004e5:	48 8b 44 24 78	mov	0x78(%rsp),%rax	E. 120
4004ea:	48 89 87 88 00 00 00	mov	%rax,0x88(%rdi)	F. 136
4004f1:	48 8b 84 24 f8 01 00	mov	0x1f8(%rsp),%rax	G. 504
4004f8:	00			
4004f9:	48 03 44 24 08	add	0x8(%rsp),%rax	
4004fe:	48 89 84 24 c0 00 00	mov	%rax,0xc0(%rsp)	H. 192
400505:	00			
400506:	48 8b 44 d4 b8	mov	-0x48(%rsp,%rdx,8),%rax	I. -72

Solution to Problem 2.19 (page 107)

The functions $T2U$ and $U2T$ are very peculiar from a mathematical perspective. It is important to understand how they behave.

We solve this problem by reordering the rows in the solution of Problem 2.17 according to the two's-complement value and then listing the unsigned value as the result of the function application. We show the hexadecimal values to make this process more concrete.

$\vec{x}$ (hex)	x	$T2U_4(vecx)$
0xF	-1	15
0xB	-5	11
0xA	-6	10
0xC	-4	12
0x1	1	1
0x8	8	8

Solution to Problem 2.20 (page 109)

This exercise tests your understanding of Equation 2.5.

For the first four entries, the values of x are negative and $T2U_4(x) = x + 2^4$. For the remaining two entries, the values of x are nonnegative and $T2U_4(x) = x$.

Solution to Problem 2.21 (page 112)

This problem reinforces your understanding of the relation between two's-complement and unsigned representations, as well as the effects of the C promotion rules. Recall that $TMin_{32}$ is $-2,147,483,648$, and that when cast to unsigned it becomes $2,147,483,648$. In addition, if either operand is unsigned, then the other operand will be cast to unsigned before comparing.

Expression	Type	Evaluation
$-2147483647-1 == 2147483648U$	Unsigned	1
$-2147483647-1 < 2147483647$	Signed	1
$-2147483647-1U < 2147483647$	Unsigned	0
$-2147483647-1 < -2147483647$	Signed	1
$-2147483647-1U < -2147483647$	Unsigned	1

Solution to Problem 2.22 (page 115)

This exercise provides a concrete demonstration of how sign extension preserves the numeric value of a two's-complement representation.

$$\begin{array}{llll} \text{A.} & [1100] & -2^3 + 2^2 & = -8 + 4 = -4 \\ \text{B.} & [11100] & -2^4 + 2^3 + 2^2 & = -16 + 8 + 4 = -4 \\ \text{C.} & [111100] & -2^5 + 2^4 + 2^3 + 2^2 & = -32 + 16 + 8 + 4 = -4 \end{array}$$

Solution to Problem 2.23 (page 116)

The expressions in these functions are common program “idioms” for extracting values from a word in which multiple bit fields have been packed. They exploit the zero-filling and sign-extending properties of the different shift operations. Note carefully the ordering of the cast and shift operations. In `fun1`, the shifts are performed on unsigned variable `word` and hence are logical. In `fun2`, shifts are performed after casting `word` to `int` and hence are arithmetic.

A.	w	fun1(w)	fun2(w)
	0x00000076	0x00000076	0x00000076
	0x87654321	0x00000021	0x00000021
	0x000000C9	0x000000C9	0xFFFFFFC9
	0xEDCBA987	0x00000087	0xFFFFF87

- B. Function `fun1` extracts a value from the low-order 8 bits of the argument, giving an integer ranging between 0 and 255. Function `fun2` extracts a value from the low-order 8 bits of the argument, but it also performs sign extension. The result will be a number between -128 and 127 .

Hex		Unsigned		Two's complement		t not for two's- sing very small
Original	Truncated	Original	Truncated	Original	Truncated	
1	1	1	1	1	1	
3	3	3	3	3	3	
5	5	5	5	5	5	
C	4	12	4	-4	4	
E	6	14	6	-2	6	

As Equation 2.9 states, the effect of this truncation on unsigned values is to simply find their residue, modulo 8. The effect of the truncation on signed values is a bit more complex. According to Equation 2.10, we first compute the modulo 8 residue of the argument. This will give values 0 through 7 for arguments 0 through 7, and also for arguments -8 through -1. Then we apply function $U2T_3$ to these residues, giving two repetitions of the sequences 0 through 3 and -4 through -1.

Solution to Problem 2.25 (page 119)

This problem is designed to demonstrate how easily bugs can arise due to the implicit casting from signed to unsigned. It seems quite natural to pass parameter `length` as an unsigned, since one would never want to use a negative length. The stopping criterion `i <= length-1` also seems quite natural. But combining these two yields an unexpected outcome!

Since parameter `length` is unsigned, the computation `0 - 1` is performed using unsigned arithmetic, which is equivalent to modular addition. The result is then *UMax*. The `≤` comparison is also performed using an unsigned comparison, and since any number is less than or equal to *UMax*, the comparison always holds! Thus, the code attempts to access invalid elements of array `a`.

The code can be fixed either by declaring `length` to be an `int` or by changing the test of the `for` loop to be `i < length`.

Solution to Problem 2.26 (page 119)

This example demonstrates a subtle feature of unsigned arithmetic, and also the property that we sometimes perform unsigned arithmetic without realizing it. This can lead to very tricky bugs.

- A. *For what cases will this function produce an incorrect result?* The function will incorrectly return 1 when `s` is shorter than `t`.
- B. *Explain how this incorrect result comes about.* Since `strlen` is defined to yield an unsigned result, the difference and the comparison are both computed using unsigned arithmetic. When `s` is shorter than `t`, the difference `strlen(s) - strlen(t)` should be negative, but instead becomes a large, unsigned number, which is greater than 0.
- C. *Show how to fix the code so that it will work reliably.* Replace the test with the following:

```
return strlen(s) > strlen(t);
```


Solution to Problem 2.27 (page 125)

This function is a direct implementation of the rules given to determine whether or not an unsigned addition overflows.

```
/* Determine whether arguments can be added without overflow */
int uadd_ok(unsigned x, unsigned y) {
    unsigned sum = x+y;
    return sum >= x;
}
```

Solution to Problem 2.28 (page 125)

This problem is a simple demonstration of arithmetic modulo 16. The easiest way to solve it is to convert the hex pattern into its unsigned decimal value. For nonzero values of x , we must have $(-\overset{u}{4}x) + x = 16$. Then we convert the complemented value back to hex.

x		$-\overset{u}{4}x$	
Hex	Decimal	Decimal	Hex
1	1	15	F
4	4	12	C
7	7	9	9
A	10	6	6
E	14	2	2

x	y	$x + y$	$x + {}^t_4 y$	Case
-8	-5	-13	3	1
[1000]	[1011]	[10011]	[0011]	
-8	-8	-16	0	1
[1000]	[1000]	[10000]	[0000]	
-8	5	-3	-3	2
[1000]	[0101]	[11101]	[1101]	
2	5	7	7	3
[0010]	[0101]	[00111]	[0111]	
5	5	10	-6	4
[0101]	[0101]	[01010]	[1010]	

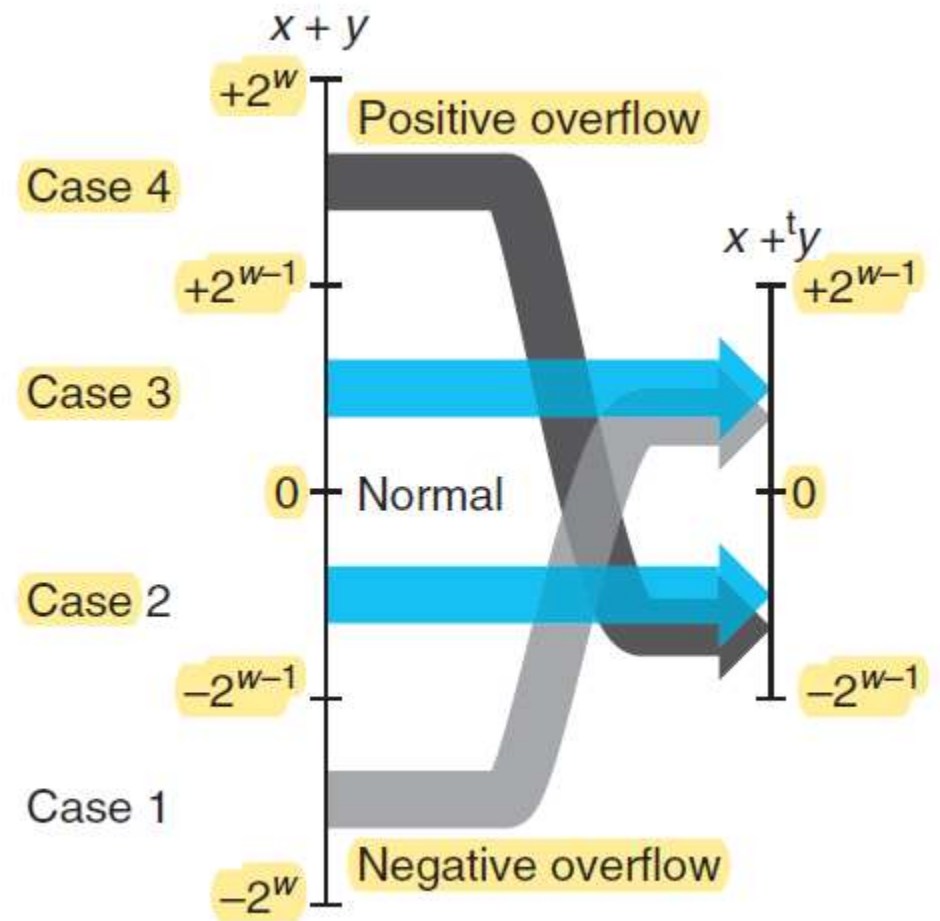
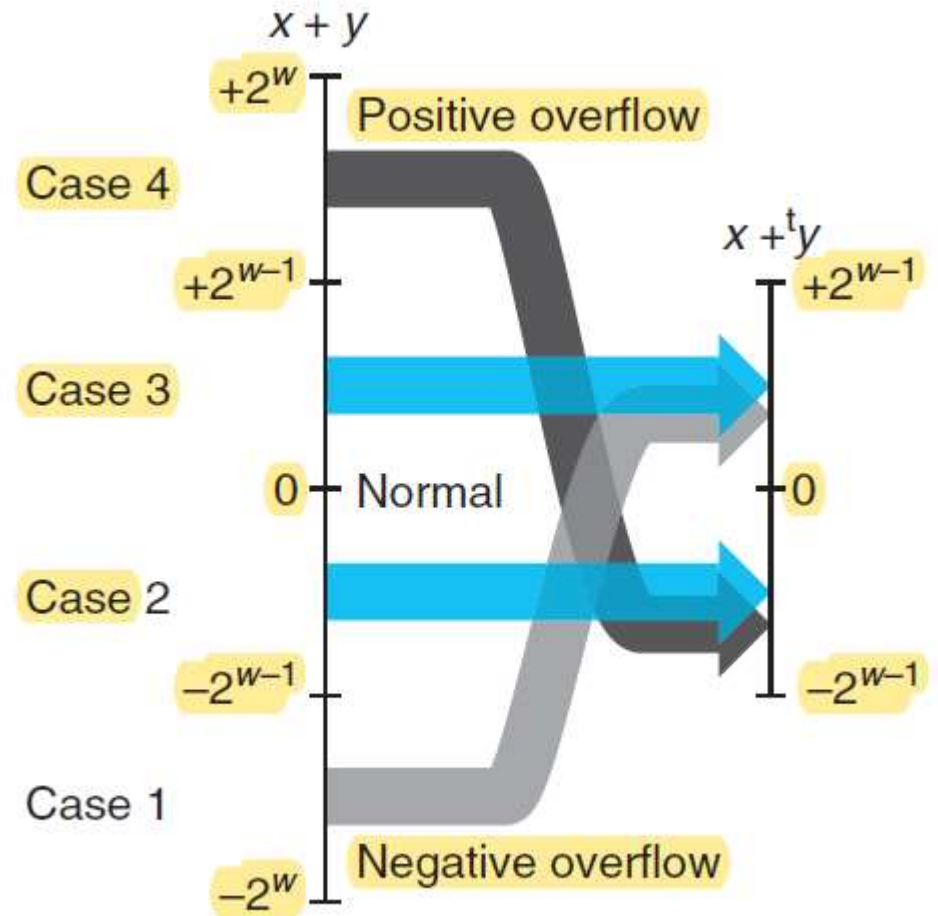


Figure 2.25 Two's-complement addition examples. The bit-level representation of the 4-bit two's-complement sum can be obtained by performing binary addition of the operands and truncating the result to 4 bits.

Solution to Problem 2.29 (page 129)

This problem is an exercise to make sure you understand two's-complement addition.

x	y	$x + y$	$x +_5^t y$	Case
-12	-15	-27	5	1
[10100]	[10001]	[100101]	[00101]	
-8	-8	-16	-16	2
[11000]	[11000]	[110000]	[10000]	
-9	8	-1	-1	2
[10111]	[01000]	[111111]	[11111]	
2	5	7	7	3
[00010]	[00101]	[000111]	[00111]	
12	4	16	-16	4
[01100]	[00100]	[010000]	[10000]	



Solution to Problem 2.30 (page 130)

This function is a direct implementation of the rules given to determine whether or not a two's-complement addition overflows.

```
/* Determine whether arguments can be added without overflow */
int tadd_ok(int x, int y) {
    int sum = x+y;
    int neg_over = x < 0 && y < 0 && sum >= 0;
    int pos_over = x >= 0 && y >= 0 && sum < 0;
    return !neg_over && !pos_over;
}
```

Solution to Problem 2.31 (page 130)

Your coworker could have learned, by studying Section 2.3.2, that two's-complement addition forms an abelian group, and so the expression $(x+y)-x$ will evaluate to y regardless of whether or not the addition overflows, and that $(x+y)-y$ will always evaluate to x .

Solution to Problem 2.32 (page 130)

This function will give correct values, except when y is $TMin$. In this case, we will have $-y$ also equal to $TMin$, and so the call to function `tadd_ok` will indicate overflow when x is negative and no overflow when x is nonnegative. In fact, the opposite is true: `tsub_ok( $x$ ,  $TMin$ )` should yield 0 when x is negative and 1 when it is nonnegative.

One lesson to be learned from this exercise is that $TMin$ should be included as one of the cases in any test procedure for a function.

Solution to Problem 2.33 (page 131)

This problem helps you understand two's-complement negation using a very small word size.

For $w = 4$, we have $TMin_4 = -8$. So -8 is its own additive inverse, while other values are negated by integer negation.

x		$-_4^t x$	
Hex	Decimal	Decimal	Hex
2	2	-2	E
3	3	-3	D
9	-9	-9	7
B	-5	5	5
C	-4	4	4

The bit patterns are the same as for unsigned negation.

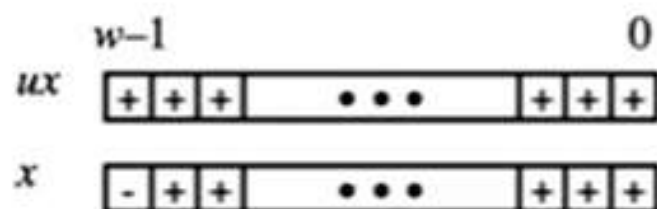
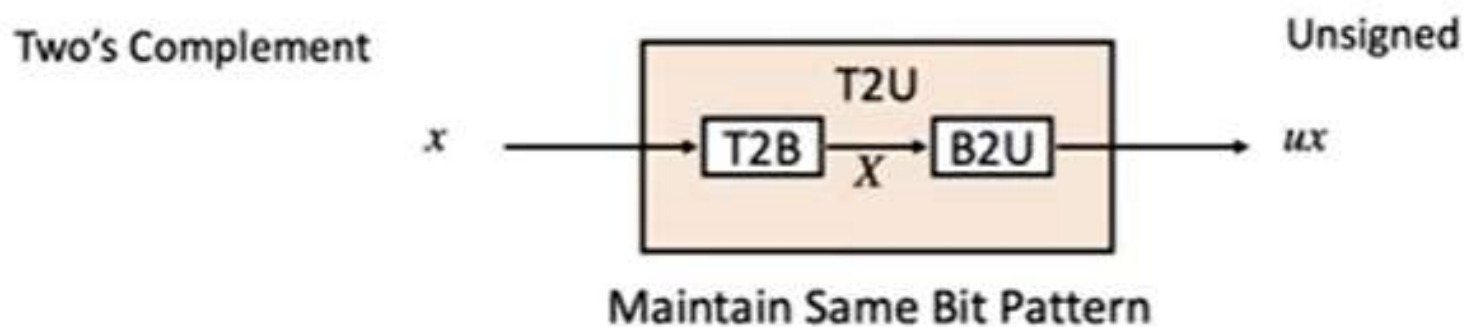
2.34

Mode	x		y		$x \cdot y$		Truncated $x \cdot y$	
Unsigned	4	[100]	5	[101]	20	[010100]	4	[100]
Two's complement	-4	[100]	-3	[101]	12	[001100]	-4	[100]
Unsigned	2	[010]	7	[111]	14	[001110]	6	[110]
Two's complement	2	[010]	-1	[111]	-2	[111110]	-2	[110]
Unsigned	6	[110]	6	[110]	36	[100100]	4	[100]
Two's complement	-2	[110]	-2	[110]	4	[000100]	-4	[100]

Mapping Signed $\leftrightarrow$ Unsigned

Bits	Signed		Unsigned
0000	0	$=$	0
0001	1		1
0010	2		2
0011	3		3
0100	4		4
0101	5		5
0110	6		6
0111	7		7
1000	-8	± 16	8
1001	-7		9
1010	-6		10
1011	-5		11
1100	-4		12
1101	-3		13
1110	-2		14
1111	-1		15

Relation between Signed & Unsigned



Large negative weight
becomes
Large positive weight

arbitrary-precision arithmetic

- In [computer science](#), **arbitrary-precision arithmetic**, also called **bignum arithmetic**, **multiple-precision arithmetic**, or sometimes **infinite-precision arithmetic**, indicates that [calculations](#) are performed on numbers whose [digits](#) of [precision](#) are limited only by the available [memory](#) of the host system. This contrasts with the faster [fixed-precision arithmetic](#) found in most [arithmetic logic unit](#) (ALU) hardware, which typically offers between 8 and 64 [bits](#) of precision.